Aran Learn 16: 3D Graphics with three.js

Aran Learn 16: 3D Graphics with three.js

For: everyone who will touch the 3D scenes in the tour. Inbathamizhan owns the 3D firewall scene; Bharadhwaj and Arun own the 3D immune space. **Time:** about 4 hours with practice. **Before this:** Learn 11 (React) and Learn 13 (SVG, animation, `useStepper`).

The tour now has two 3D scenes. In the first, live requests fly as glowing packets from bank systems into a 3D Aran shield and out to the systems that store them. In the second, you can turn the immune layer's real feature space in your hands and see its normal zone and its detectors. Both use **three.js**, the most widely used 3D library for the web. This doc teaches the handful of ideas behind them.

1. What a 3D scene is made of

Every three.js program has the same five parts:

Part	What it is	Aran example
Scene	the container for everything you can see	`new THREE.Scene()`
Camera	where you look from; `PerspectiveCamera` makes far things smaller, like your eye	camera at (0, 4, 16), looking at the shield
Renderer	draws the scene into a `<canvas>` using the graphics card (WebGL)	`new THREE.WebGLRenderer({ antialias: true, alpha: true })`
Mesh	a shape (**geometry**) plus a look (**material**)	the shield is an `IcosahedronGeometry` with a glowing material
Light	without light, standard materials are black	one soft `AmbientLight` and one `PointLight` inside the shield

```
import * as THREE from 'three'
const scene = new THREE.Scene()
const camera = new THREE.PerspectiveCamera(45, width / height, 0.1, 100)
camera.position.set(0, 4, 16)
const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true })
renderer.setSize(width, height)
container.appendChild(renderer.domElement)

const shield = new THREE.Mesh(new THREE.IcosahedronGeometry(1.6, 1),
```

```
          new THREE.MeshStandardMaterial({ color: 0x1f8a8a, emissive: 0x0b3d5c, flatShading:
true }))
        scene.add(shield, new THREE.AmbientLight(0xffffff, 0.6))
        renderer.render(scene, camera)
```

Coordinates. x goes right, y goes up, z comes towards you. Units are whatever you decide; in Aran one unit is roughly the size of a node box.

2. The animation loop

A 3D scene is redrawn about 60 times a second. `requestAnimationFrame` (Learn 13, section 5) calls your function before each screen refresh:

```
        const clock = new THREE.Clock()
        function frame() {
          const t = clock.getElapsedTime()
          shield.rotation.y = t * 0.4            // slow spin
          renderer.render(scene, camera)
          raf = requestAnimationFrame(frame)
        }
        frame()
```

Inside React, start this loop in `useEffect` and **always** clean up: cancel the frame, call `renderer.dispose()`, and dispose every geometry and material. Otherwise each visit to the chapter leaks graphics memory until the browser slows down.

3. Moving things along a path

Packets follow smooth curves. three.js has `CatmullRomCurve3`, a curve through a list of points; `curve.getPointAt(k)` gives the position a fraction `k` (0 to 1) along it.

```
        const path = new THREE.CatmullRomCurve3([source.position, new THREE.Vector3(-3, 1,
0), shield.position])
        packet.position.copy(path.getPointAt(Math.min(1, elapsed / 1.2)))
```

When the packet reaches the shield we swap its material from red (raw customer data) to amber (Aran token) and send it along the second curve to the sink. This is exactly what the server did to the request: the colours come from the real response's findings.

4. Letting people turn the scene

`OrbitControls` (from `three/examples/jsm/controls/OrbitControls.js`) lets the viewer drag to rotate, scroll to zoom and right-drag to pan. One line, and a flat picture becomes something a reviewer can explore. We set `enableDamping` so movement eases out instead of stopping dead, and `autoRotate` so the scene drifts gently until someone touches it.

5. Labels that stay readable

Text drawn inside WebGL blurs and is hard to make accessible. `CSS2DRenderer` places ordinary HTML elements at 3D positions, so labels stay crisp, use our fonts and theme colours, and can be read by screen readers.

6. The 3D immune space: why the picture is honest

The immune layer measures seven features per traffic window (Learn 6). We cannot draw seven dimensions, so the scene lets you pick **three** of them as the x, y and z axes and drops the rest. This is an *orthogonal projection*, and it has a property that makes the picture trustworthy:

- A **detector** is a ball in seven dimensions (a centre and a radius). The shadow of a ball on any three axes is a ball with **the same radius**. So the translucent red spheres you see are the exact shadows of the real detectors.
- The **normal zone** the detector learned is the box from 0.1 to 0.7 on every axis (the scaler maps the range seen while learning to that interval). Its shadow is the green box.
- A traffic window that sits inside a detector in seven dimensions also sits inside that detector's shadow. The reverse is not always true, so a blue dot can appear inside a red sphere's shadow without being an alert. The dot's colour always comes from the real score.

7. Keeping it light

- Lazy-load three.js (`React.lazy(() => import('./three/Firewall3D.jsx'))`) so pages without 3D stay fast.
- Cap the pixel ratio at 2, draw at most a few hundred spheres, and use low-detail geometry for small objects.
- Pause the loop when the tab is hidden, and render nothing at all if WebGL is unavailable: the 2D scenes from Learn 13 remain as the fallback.
- With "reduce motion" turned on, the scene does not auto-rotate.

Practice

1. **Hello cube.** Make a scene with one spinning cube and a light. Change its colour to `var(--accent)` by reading the CSS variable with `getComputedStyle(document.documentElement).getPropertyValue('--accent')`.
2. **Orbit.** Add `OrbitControls` and confirm you can drag the cube around.
3. **A packet.** Draw a `CatmullRomCurve3` through three points and move a small sphere along it in 2 seconds, then back.
4. **Real data.** Call `POST /api/tour/packet` and make the sphere red if `findings` contains a protected value, grey otherwise.
5. **Projection.** Call `GET /api/tour/immune-space?axes=0,1,3` and draw each self point as a small green sphere. Check that every one lies inside the 0.1 to 0.7 box.

Solutions

1. `new THREE.Color(getComputedStyle(document.documentElement).getPropertyValue('--accent').trim())` gives a three.js colour from the theme token.
2. `const controls = new OrbitControls(camera, renderer.domElement);` call `controls.update()` inside the loop.
3. `const k = (t % 4) / 2; sphere.position.copy(curve.getPointAt(k <= 1 ? k : 2 - k)).`
4. `const raw = ev.findings.some(f => f.protected); sphere.material.color.set(raw ? 0xc2362f : 0x888888).`
5. Each point `p` satisfies `0.1 <= p[i] <= 0.7` for the chosen axes, because the scaler maps the learning range there.